Übungsblatt 2: RISC-V Basics & Venus Onboarding

Mikrocomputertechnik 1

Musterlösung

Zielsetzung

In diesem Übungsblatt lernen Sie die Philosophie der RISC-V-Architektur kennen und schreiben Ihre ersten eigenen Programme. Sie machen sich mit den ABI-Registernamen vertraut, verstehen den Unterschied zwischen R-Type- und I-Type-Befehlen und nutzen den Venus-Simulator für schrittweises Debugging (Tracing).

Aufgabe 1: Architektur und ABI

Bevor wir Code schreiben, müssen die Spielregeln der Hardware sitzen. Beantworten Sie die folgenden Fragen anhand der Vorlesung.

Load/Store-Architektur

RISC-V ist eine strikte Load/Store-Architektur. Was bedeutet das für Berechnungen in der ALU in Bezug auf den Arbeitsspeicher (RAM)?

Die ABI (Application Binary Interface)

Warum sollten Sie im Assembler-Code ABI-Namen (wie `t0`, `a0`) anstelle der reinen Hardware-Namen (wie `x5`, `x10`) nutzen?

Das Zero-Register

Was passiert physikalisch, wenn Sie versuchen, das Ergebnis einer Addition in das Register `x0` (Zero) zu speichern?

Ihre Lösung

(Notieren Sie Ihre Antworten hier.)

Musterlösung Aufgabe 1

Load/Store-Architektur:

- Die ALU kommuniziert nicht direkt mit dem Arbeitsspeicher (RAM).
- Berechnungen finden ausschließlich zwischen Registern statt.
- Daten müssen erst per Load aus dem RAM geholt und Ergebnisse später per Store zurückgeschrieben werden.

ABI:

- Die Hardware erzwingt die ABI nicht; sie ist ein Vertrag für die Software.
- Bibliotheken und Betriebssystem-Aufrufe (Syscalls) erwarten Parameter in festgelegten Registern (z. B. Venus: Dienstcode in `a0`).
- Ohne Einhaltung der ABI bricht die Kompatibilität.

Zero-Register:

- `x0` ist fest auf 0 verdrahtet (Hardwired Zero).
- Schreibversuche werden von der Hardware ignoriert; der Wert bleibt 0.

Aufgabe 2: Immediates und die 12-Bit-Grenze

Wir wollen Konstanten in unseren Code einbauen (I-Type-Befehle).

Konstanten-Limit

Sie möchten den Wert 2048 in das Register `t0` laden. Warum schlägt der Befehl `addi t0, x0, 2048` fehl? Welche Alternative (Pseudo-Instruktion) sollten Sie stattdessen nutzen?

Fehlende Befehle

In der RISC-V-Basisarchitektur (RV32I) gibt es `sub` (Subtraktion zweier Register), aber keinen Befehl `subi` (Subtraktion einer Konstante). Warum haben die Chip-Designer diesen Befehl weggelassen?

Ihre Lösung

(Notieren Sie Ihre Antworten hier.)

Musterlösung Aufgabe 2

Konstanten-Limit:

- Beim I-Type stehen für das Immediate exakt 12 Bit zur Verfügung.
- Als vorzeichenbehaftete Zahl im Zweierkomplement reicht der Bereich von -2048 bis $+2047$.
- Die Zahl 2048 sprengt dieses Limit.
- Alternative: Pseudo-Instruktion `li t0, 2048` (Load Immediate); der Assembler übersetzt sie in gültige Maschinenbefehle.

Fehlende `subi`:

- Die Subtraktion einer Konstanten K ist identisch zur Addition von $-K$.
- Stattdessen reicht `addi` mit negativem Immediate (z. B. `addi t0, t0, -7`).
- RISC vermeidet redundante Befehle, um die Hardware schlank zu halten.

Aufgabe 3: Venus Onboarding — Die erste Gleichung

Öffnen Sie den Venus-Simulator. Setzen Sie die Mathematik-Aufgabe aus der Vorlesung um.

Gegeben ist die Gleichung

$$Y = (A + B) - (C + D)$$

mit den Startwerten $A = 5$, $B = 10$, $C = 3$, $D = 2$.

Aufgabenstellung

1. Nutzen Sie das `.text`-Segment und das Label `main:` als Einstiegspunkt.
2. Laden Sie die vier Konstanten über Pseudo-Instruktionen (`li`) in die temporären Register `t0` bis `t3`.
3. Berechnen Sie die beiden Klammern (Zwischensummen) und speichern Sie diese in `t4` und `t5`.
4. Subtrahieren Sie die Zwischensummen und speichern Sie das finale Ergebnis (Y) in `t6`.
5. Simulations-Check: Klicken Sie sich mit dem Step-Button durch den Code. Verifizieren Sie nach dem letzten Schritt, ob in `t6` der Wert 10 steht.
6. Konsolenausgabe (Venus-ABI): Nutzen Sie `ecall`, um das Ergebnis auszugeben. Laden Sie den Dienstcode 1 in `a0` und den Wert aus `t6` in `a1`.

Ihre Lösung

(Fügen Sie Ihren Assembler-Code hier ein.)

Musterlösung Aufgabe 3

```
.globl main
.text
main:
    # 1. Konstanten laden (A bis D in t0 bis t3)
    li t0, 5        # A = 5
    li t1, 10       # B = 10
    li t2, 3        # C = 3
    li t3, 2        # D = 2

    # 2. Zwischensummen berechnen
    add t4, t0, t1  # t4 = A + B (15)
    add t5, t2, t3  # t5 = C + D (5)

    # 3. Finales Ergebnis
    sub t6, t4, t5  # t6 = 15 - 5 = 10

    # 4. Konsolenausgabe (Venus: ID in a0, Argument in a1)
    li a0, 1        # Print Integer
    add a1, x0, t6  # t6 nach a1 kopieren (x0-Trick)
    ecall
```

Aufgabe 4: Bit-Shifting und der x0-Trick

Die ALU der Basisarchitektur (RV32I) besitzt keine Hardware-Einheit für Multiplikation. Schiebeoperationen (Shifts) multiplizieren jedoch schnell mit Potenzen von 2.

Aufgabenstellung in Venus

1. Laden Sie die Zahl 5 in das Register `t1`.
2. Multiplizieren Sie diese Zahl mit 8, indem Sie einen einzigen `slli`-Befehl (Shift Left Logical Immediate) anwenden und das Ergebnis in `t2` speichern. (Tipp: $2^3 = 8$ — um wie viele Bitstellen schieben?)
3. Negieren Sie das Ergebnis in `t2` mathematisch (aus $+40$ wird -40) und speichern Sie es in `t3`. Nutzen Sie dafür `sub` und den Trick mit dem Zero-Register (`x0`).

Ihre Lösung

(Fügen Sie Ihren Assembler-Code hier ein.)

Musterlösung Aufgabe 4

```
.text
main:
    # 1. Startwert laden
    li t1, 5

    # 2. Multiplikation mit 8 durch Shift nach links
    # Shift um 1 Bit = *2; Shift um 3 Bits = *8
    slli t2, t1, 3      # t2 = 5 * 8 = 40

    # 3. Negieren über den x0-Trick (0 - X = -X)
    sub t3, x0, t2      # t3 = 0 - 40 = -40
```

Aufgabe 5: Maschinencode von Hand

In der Vorlesung haben Sie das R-Type-Format (opcode, rd, funct3, rs1, rs2, funct7) kennengelernt. Für die Kodierung nutzen wir bewusst die Hardware-Namen `x…`, weil sie den 5-Bit-Feldern entsprechen.

Assemblieren

Assemblieren Sie den Befehl `add x9, x20, x21` von Hand zu einem 32-Bit-Wort (Hexadezimal). Nutzen Sie: opcode `0110011`, funct3 `000`, funct7 `0000000`.

Decodieren

Decodieren Sie das Maschinenwort `0x007302B3`. Welcher Assembler-Befehl (mit ABI-Namen) steckt dahinter?

Ihre Lösung

(Notieren Sie Ihre Rechenwege hier.)

Musterlösung Aufgabe 5

Assemblieren `add x9, x20, x21`:

- funct7 = `0000000`, rs2 = `10101` (21), rs1 = `10100` (20), funct3 = `000`, rd = `01001` (9), opcode = `0110011`
- Binär: `0000000 10101 10100 000 01001 0110011`
- Hex: **`0x015A04B3`**

Decodieren `0x007302B3`:

- Binär: `0000000 00111 00110 000 00101 0110011`
- funct7=`0000000`, rs2=`x7`=`t2`, rs1=`x6`=`t1`, funct3=`000`, rd=`x5`=`t0`, opcode R-Type add
- Befehl: **`add t0, t1, t2`**